sixth

The part of the quant research loop that remembers.

How to use it

A hedge fund is one cycle repeating: **research** → **code** → **backtest** → **live** → **post-mortem** → **fine-tune**. Five of those six work today — you can describe a strategy in English and have an agent write, test and deploy it in about ninety seconds.

The sixth doesn't work, and it isn't hard. It's just missing. The lesson from a losing test lands in a log file and stays there; the next strategy your agent writes has never heard of it. Every cycle restarts from the same place, so nothing compounds.

sixth is that missing part: a persistent, queryable graph of what you tested, what failed, *in which market regime*, and what you have never tried at all.

> **Sixty seconds from here to a full research cycle**
>
> ```
> git clone https://github.com/willfuse/sixth && cd sixth
> pip install -e .
> sixth demo
> ```
>
> No API key, no data vendor, no network. Zero dependencies — pure Python standard library, 3.9 and up.

What you run	What you get back
sixth add	a hypothesis stated, checked against near-duplicates you already tested
sixth prereg	your expectation hashed and sealed, before any result exists
sixth run	walk-forward, an adversarial breaker pass, a sealed-holdout verdict
sixth context	everything learned, formatted to paste into your next agent prompt

Built from the argument in antpalkin's article on the self-improving trading machine, which identifies the fine-tune step as the piece nobody has assembled end to end. Research tooling only — it places no orders and is not investment advice. See section 8.

1 Install and first run

You need Python 3.9 or newer. Nothing else — there is no numpy, no pandas, no vendor SDK. The whole package is standard library, so it installs anywhere and runs offline.

```
git clone https://github.com/willfuse/sixth
cd sixth
python3 -m venv .venv && source .venv/bin/activate
pip install -e .

sixth demo          # the entire loop, end to end, on synthetic data
```

sixth demo states three hypotheses, seals an expectation for each, runs a full cycle on all three, and prints the research context that accumulated. Read its output once before anything else — it is the whole argument of the package in about ninety lines.

Every command takes `--db` to pick a graph file. It is a top-level flag, so it goes *before* the subcommand:

```
sixth --db research.sqlite status        # correct
sixth status --db research.sqlite        # error: unrecognized arguments

export SIXTH_DB=~/research.sqlite        # or set it once
```

The default is `sixth.sqlite` in the current directory. One file holds everything: hypotheses, sealed expectations, every result, every lesson. Back it up like source code — it is the asset.

2 The workflow, in four commands

A research cycle is always these four steps in this order. The order is enforced: you cannot run a test that has no sealed expectation.

Step 1 — State the hypothesis

```
sixth add "A 20/100 moving-average cross captures trend persistence \
           and beats buy-and-hold risk-adjusted." --tags trend,equity
```

This runs nothing. Stating an idea and testing it are separate acts, which is why **sixth frontier** can later tell you what you thought of but never got to. If the statement closely matches something already in the graph, the command refuses and shows you the earlier one — override with `--force` if it really is different.

Step 2 — Seal what you expect, before you look

```
sixth prereg a-20-100-moving-average-cross-captures-trend-persistence-and \
    --must "sharpe >= 0.8" \
    --must "max_drawdown <= 0.25" \
    --rationale "trends persist long enough that a lagging filter still catches them"
```

Borrowed from clinical trials. The conditions are hashed, timestamped and written to the database, which then refuses to update or delete the row. The verdict is computed by machine against this record, so you cannot move the goalposts on a strategy that preregistered its own thesis.

--must conditions decide confirmed versus refuted. **--should** conditions are secondary: failing one alone downgrades the result to *inconclusive* rather than refuting it. At least one **--must** is required, because an expectation with nothing falsifiable in it is not an expectation.

Step 3 — Run the cycle

```
sixth run a-20-100-moving-average-cross-captures-trend-persistence-and \
    --strategy sma_cross \
    --grid fast=10,20,40 slow=60,100,200 \
    --create-proposals
```

One command does all of this, in order:

- **Seals a holdout.** The last 20% of the data is split off and physically not handed to anything that fits parameters.
- **Walks forward** across the rest with a purge and embargo between each training and test window, so a 100-bar indicator can never train on its own test set.
- **Counts every trial.** Nine parameter combinations over five folds is 45 trials, not one.
- **Runs the breaker** — an adversary that only tries to kill the strategy.
- **Looks at the sealed holdout once**, with the parameters now frozen.
- **Writes the autopsy, the verdict and the lessons** into the graph, and proposes the follow-up hypotheses the failure implies.

Step 4 — Hand it all to your next agent

```
sixth context --focus "mean reversion on intraday bars"
```

This is the step that closes the loop, and the reason the other three are worth doing. Page 6 covers it.

3 Reading what comes back

A cycle prints four blocks. Here is a real one, with what each number is telling you.

```
VERDICT: REGIME_CONDITIONAL   (experiment #1)

WALK-FORWARD (development data)
  out-of-sample Sharpe 0.162
  Sharpe decay IS->OOS 0.183
  parameter stability  80%
  folds profitable     80%

BREAKER (adversarial)
  probes survived      56% (10/18)
  edge dies at         110 bps round-turn
  vs random null       p = 0.129

SEALED HOLDOUT (touched once, just now)
  Sharpe               0.567
  max drawdown         21.9%
  Deflated Sharpe      0.072 over 45 trials
```

Line	What it means	Worry when
out-of-sample Sharpe	performance on windows never used to fit parameters	below your must-condition
Sharpe decay	in-sample minus out-of-sample, averaged over folds	above ~0.5: fitting noise
parameter stability	share of folds that chose the same parameters	below 50%: nothing stable to deploy
probes survived	share of adversarial probes the strategy lived through	any kill you can't explain
edge dies at	round-turn cost that zeroes the edge	near your real execution cost
vs random null	p-value against exposure-matched coin flips	above 0.05: indistinguishable from chance
Deflated Sharpe	probability the edge survives the search that found it	below ~0.5: the search produced it

The five verdicts

Verdict	Meaning
confirmed	every must-condition held, and every should-condition too
regime_conditional	failed overall, but held completely inside at least one regime — usually the most useful outcome you can get
refuted	failed its must-conditions everywhere, in every regime
inconclusive	must-conditions held but a should-condition failed, or nothing falsifiable was stated
retired	you set it aside deliberately; kept so it is never re-tried by accident

> **Why a refuted result is worth as much as a confirmed one**
>
> Both cost the same to produce, and only one of them is usually kept. A refutation with a regime attached tells your next agent where not to spend its next forty attempts. That is the asset this package exists to stop you throwing away.

4 The number the graph makes honest

A Sharpe ratio does not tell you whether an edge is real. It tells you what the best of however many things you tried happened to score. Search four thousand parameter combinations against noise and one of them will look excellent.

The Deflated Sharpe Ratio corrects for exactly this — but it needs an input almost nobody keeps: **how many variants you actually looked at**. The graph watched every one, so it can supply the real number instead of a flattering one.

Trials the graph has watched	Deflated Sharpe	Reading
1 (what you would report)	0.930	looks like a strong edge
45 (this search alone)	0.072	probably the search, not the market
90 (after the next cycle)	0.024	the bar has risen
105 (after the one after that)	0.003	nothing here survives the count

Same strategy, same data, same returns. Only the honest trial count changes. This is what a research programme with a memory looks like from the inside: the standard of proof rises as you spend your search budget, which is what should happen and never does when the count is discarded after each run.

5 Asking the graph questions

Once a few cycles have run, the graph answers things no log file can.

Command	Question it answers
sixth status	how big is this programme, and how many trials has it spent
sixth list --status refuted	what have I already ruled out
sixth frontier	what did I think of and never actually test
sixth regime down/stressed	what is known to bleed in a falling, volatile market — including strategies that pass overall
sixth show <slug>	everything about one idea: every run, every regime, every lesson, what it was derived from
sixth lessons --kind overfit	every time this programme has fooled itself the same way
sixth export	the whole graph as JSON, for diffing or feeding to a model

The regime query is the one worth learning first. Results are stored sliced by market condition — trend direction crossed with volatility level, both computed causally from trailing windows only. So "this failed" becomes "this failed in `down/stressed` and made all of its money in `up/calm`, which was 18% of the sample", which is an actionable research direction rather than a dead end.

```
$ sixth regime down/normal

2 hypotheses lose money in down/normal

  a-20-100-moving-average-cross...   sharpe -2.82 over 355 bars
    A 20/100 moving-average cross captures trend persistence...
  simple-60-day-time-series-momentum...  sharpe -3.61 over 84 bars
    Simple 60-day time-series momentum earns a positive Sharpe...
```

6 Closing the loop

Everything so far is bookkeeping. This is the part that makes it compound.

sixth context renders the entire graph as a markdown block designed to sit at the top of a prompt. Paste it above your next request to Claude Code, or pipe it into whatever writes your strategies:

```
sixth context --focus "intraday mean reversion" | pbcopy
sixth context --json | jq '.frontier'      # for programmatic use
```

What it emits:

```
## Prior research state (do not re-derive)

This graph holds 13 hypotheses across 3 recorded experiments and 105
total trials. Treat everything below as already established.

### Works only in specific regimes
- a-20-100-moving-average-cross...
  - holds in: up/calm, up/normal, up/stressed
  - bleeds in: down/calm, flat/calm, flat/normal

### Recurring failure patterns in this research programme
- [overfit] Sharpe decays 0.54 from in-sample to out-of-sample.
- [execution] Edge disappears at roughly 22 bps round-turn cost.
- [concentration] Removing the best regime leaves Sharpe -0.10.

### Frontier - stated but never tested
- ...traded only during up/calm, up/normal, up/stressed

### Rules for the next proposal
1. It must not duplicate anything under Refuted.
2. State it as a falsifiable claim with numeric must-conditions.
3. Assume 105 trials have already been spent; the Deflated Sharpe
   bar rises with every additional one.
```

The agent writing your next strategy now starts from everything the last forty attempts learned, instead of from zero. It knows which ideas are spent, which regimes eat this style of signal, and how many trials the programme has already burned. That is the difference between a trading bot and something that compounds.

`examples/agent_loop.py` in the repo shows the full pattern, including the near-duplicate check that stops an agent proposing something the graph has already settled.

7 Your own data and your own strategies

Real bars

```
sixth run my-hypothesis --strategy sma_cross \
    --csv ~/data/SPY.csv --symbol SPY \
    --commission-bps 1 --slippage-bps 3
```

Any CSV with `date` and `close` columns works; `open`, `high`, `low` and `volume` are used when present. With no `--csv`, a deterministic regime-switching synthetic series is generated so everything runs offline. That series is a *test fixture with known ground truth*, not a market — point it at real bars before believing any number.

Every stored result carries a fingerprint of the data it ran on, so a result can never be silently compared against a different dataset.

Writing a strategy

A strategy is a function from bars to target weights in `[-1, 1]`. That is the entire contract:

```
from sixth import register

@register("breakout", "Long when today closed at a 50-bar high.", window=50)
def breakout(bars, window=50):
    return [1.0 if i >= window and bars.close[i] >= max(bars.close[i-window:i])
            else 0.0
            for i in range(len(bars))]
```

The engine computes weight `i` from data through bar `i` and fills it at bar `i+1`'s open. Lookahead is structurally impossible rather than merely discouraged — a strategy literally cannot trade the bar that produced its own signal. There is a test that proves it, and another that confirms a peeking strategy dies the moment a one-bar delay is applied.

Run **sixth strategies** for the built-ins: `sma_cross`, `sma_cross_ls`, `momentum`, `mean_reversion`, `buy_hold`, `flat` and `random_signal`. The last three are null models — the things your idea has to beat.

As a library

```
from sixth import (HypothesisGraph, Expectation, run_cycle,
                   synthetic_bars, brief)

graph = HypothesisGraph("research.sqlite")
h = graph.add("A 20/100 MA cross beats buy-and-hold.", tags=["trend"])
graph.preregister(h.id, Expectation.parse(["sharpe >= 0.8",
                                           "max_drawdown <= 0.25"]))

result = run_cycle(graph, h.id, "sma_cross", synthetic_bars(),
                   grid={"fast": [10, 20, 40], "slow": [60, 100, 200]},
                   create_proposals=True)

print(result.verdict)      # 'regime_conditional'
print(brief(graph))        # the block for your next agent prompt
```

8 The risk gate, and what this will not do

The kill switch lives in code

A risk limit written into a system prompt is a suggestion, and a sufficiently motivated reasoning chain will argue its way past it. So `RiskGate` has no text interface at all. It returns a smaller number, or it raises. There is no argument to make to it.

```
from sixth import RiskGate, RiskLimits
from sixth.live import Order

gate = RiskGate(RiskLimits(max_position=0.5, max_drawdown=0.10,
                           max_daily_loss=0.02, allow_shorts=False))

gate.check(Order("AAPL", 5.0))     # -> 0.25   clamped, not refused
gate.check(Order("AAPL", -1.0))    # -> 0.0    shorts disabled
gate.mark(equity=85_000)           # -15% from peak
gate.check(Order("AAPL", 0.1))     # -> KillSwitchTripped
```

Once tripped it stays tripped — a later recovery in equity does not resume trading. Every clamp is journalled with the rule that fired, which is what the post-mortem reads later.

Scope

- **It does not trade.** The package ships a paper broker and the risk gate. It routes no real orders and holds no venue credentials. `BrokerAdapter` is the interface to implement if you go live; read the whole class first, and route every order through the gate.
- **It is not investment advice.** A *confirmed* verdict is not a recommendation. It means a claim you wrote down in advance survived a set of mechanical checks on historical data.
- **Backtests are not returns.** Everything in the package — the breaker, the trial counter, the sealed holdout, the random-null test — exists to make that gap visible rather than to close it.
- **The synthetic market is a fixture.** It gives the tests known ground truth and lets the demo run anywhere. It is not a claim about real markets.

9 Where things live

Path	What it is
src/sixth/graph.py	the hypothesis graph — the sixth part, and the reason the repo exists
src/sixth/prereg.py	sealed expectations and machine-computed verdicts
src/sixth/stats.py	Sharpe, PSR, Deflated Sharpe, stationary-bootstrap Monte Carlo
src/sixth/backtest.py	the engine: costs, slippage, borrow, no lookahead
src/sixth/walkforward.py	purged and embargoed folds, plus the sealed holdout
src/sixth/breaker.py	the adversary that only tries to kill strategies
src/sixth/postmortem.py	the autopsy: where it bled, by regime and by episode
src/sixth/lessons.py	autopsies → graph rows → proposed next hypotheses
src/sixth/context.py	renders the graph as agent context — the loop closing
src/sixth/live.py	paper broker and the risk gate
examples/agent_loop.py	driving the cycle from an agent instead of the CLI
tests/	150 tests, no dependencies beyond pytest

`python -m pytest tests -q` — including the ones that matter: that a strategy cannot trade the bar that produced its own signal, that a lookahead strategy dies under a one-bar delay, that a coin flip is never certified as an edge, that training windows never touch their test windows, and that the database refuses to let you edit a sealed expectation.

---

github.com/willfuse/sixth · MIT · Built from the six-part loop described by @antpalkin. Research tooling; not investment advice.